

Répartition des blocs de code

Détection de voies routières

Maryam Boufous

1 Chargement de l'image

Description : Ce bloc charge l'image depuis le disque et vérifie qu'elle existe.

```
Mat image = imread("route.jpg");

if (image.empty()) {
    cout << "Erreur_chargement_image" << endl;
    return -1;
}
```

2 Conversion en RGB

Description : Convertit l'image BGR (format OpenCV) en RGB pour un affichage correct.

```
Mat image_rgb;
cvtColor(image, image_rgb, COLOR_BGR2RGB);
```

3 Filtrage HSV (blanc + jaune)

Description : Permet d'isoler les marquages routiers (blanc et jaune) indépendamment de la luminosité.

```
Mat hsv;
cvtColor(image, hsv, COLOR_BGR2HSV);

Mat masque_blanc, masque_jaune;

inRange(hsv, Scalar(0, 0, 200),
        Scalar(180, 50, 255),
        masque_blanc);

inRange(hsv, Scalar(15, 60, 100),
        Scalar(35, 255, 255),
        masque_jaune);

Mat masque;
bitwise_or(masque_blanc, masque_jaune, masque);
```

4 Détection de contours (Canny)

Description : Détecte les bords dans l'image pour identifier les contours des lignes.

```
Mat gris, flou, contours;

cvtColor(image, gris, COLOR_BGR2GRAY);
GaussianBlur(gris, flou, Size(5,5), 0);

Canny(flou, contours, 50, 150);
```

5 Combinaison masque + contours

Description : Garde uniquement les contours situés dans les zones de couleur détectées.

```
Mat combine;
bitwise_and(contours, masque, combine);
```

6 Transformation de perspective

Description : Transforme la vue en "bird eye view" pour rendre les voies parallèles.

```
vector<Point2f> src = {
    Point2f(largeur * 0.15, hauteur),
    Point2f(largeur * 0.45, hauteur * 0.6),
    Point2f(largeur * 0.55, hauteur * 0.6),
    Point2f(largeur * 0.85, hauteur)
};

vector<Point2f> dst = {
    Point2f(0, hauteur),
    Point2f(0, 0),
    Point2f(largeur, 0),
    Point2f(largeur, hauteur)
};

Mat M = getPerspectiveTransform(src, dst);
Mat bird;

warpPerspective(combine, bird, M, Size(largeur, hauteur));
```

7 Histogramme (détection base des voies)

Description : Trouve les positions initiales des voies gauche et droite.

```
Mat histogramme;
reduce(image_bird, histogramme, 0, REDUCE_SUM);

int milieu = histogramme.cols / 2;
```

```
Point maxG, maxD;

minMaxLoc(histogramme(Rect(0, 0, milieu, 1)), 0, 0, 0, &maxG);
minMaxLoc(histogramme(Rect(milieu, 0, milieu, 1)), 0, 0, 0, &maxD
);
```

8 Fenêtres glissantes

Description : Suit les lignes en remontant l'image et collecte les pixels appartenant aux voies.

```
for (int i = 0; i < nb_fenetres; i++) {
    // definir zone
    // recuperer pixels
    // recentrer fenetre
}
```

9 Ajustement polynomial

Description : Approxime chaque voie par une parabole.

```
solve(A, b, coeffs, DECOMP_SVD);

//  $x = A*y^2 + B*y + C$ 
```

10 Visualisation des voies

Description : Dessine les lignes et remplit la zone entre elles.

```
polylines(image, points, false, Scalar(0,255,0), 20);
fillPoly(image, zone, Scalar(0,255,0));
```

11 Projection inverse

Description : Reprojettes les voies détectées sur l'image originale.

```
warpPerspective(dessin, retour, M_inv, Size(largeur, hauteur));
addWeighted(image_rgb, 1.0, retour, 0.5, 0, resultat);
```

12 Calcul des métriques

Description : Calcule le rayon de courbure et le décalage du véhicule.

```
double R = pow(1 + pow(2*A*y + B, 2), 1.5) / abs(2*A);

double offset = (centre_image - centre_voie) * mppx;
```

13 Affichage final

Description : Affiche les informations sur l'image finale.

```
putText(resultat, "Rayon: □...", Point(50,50),  
        FONT_HERSHEY_SIMPLEX, 1, Scalar(255,255,255), 2);
```